

UW PICO 5.09

File: memory_and_test_shell.c

```
if (*str == 0) return str;
char *end = str + strlen(str) - 1;
while (end > str && (*end == ' ' || *end == '\t')) end--;
end[1] = '\0';
return str;
}

void execute_command(char *line) {
    line = trim_space(line);
    if (strlen(line) == 0) return;

    // Dynamically allocate argument array to verify leak-free execution
    char **args = calloc(MAX_ARGS, sizeof(char *));
    if (!args) {
        perror("calloc failed");
        return;
    }

    int arg_count = 0;
    char *token = strtok(line, " \t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t");
    }
    args[arg_count] = NULL;

    if (arg_count > 0) {
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork failed");
        } else if (pid == 0) {
            execvp(args[0], args);
            perror("Command execution failed");
            free(args); // Clean release in child
            exit(EXIT_FAILURE);
        } else {
            int status;
            waitpid(pid, &status, 0);
        }
    }

    // Cleanly release allocated memory resources in parent
    free(args);
}

int main() {
    char input_buf[BUF_SIZE];
```

 Get Help
 Exit

 WriteOut
 Justify

 Read File
 Where is

 Prev Pg
 Next Pg

 Cut Text
 UnCut Tex

~ -- zsh

...

~ -- nano memory

UW PICO 5.09File: memory_and_test_shell.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_ARGS 32
#define BUF_SIZE 1024

// Dynamically allocate prompt string to demonstrate dynamic memory tracking & clean release
char *create_dynamic_prompt(const char *shell_name) {
    char *prompt = malloc(strlen(shell_name) + 4);
    if (!prompt) {
        perror("malloc failed");
        exit(EXIT_FAILURE);
    }
    sprintf(prompt, "%s > ", shell_name);
    return prompt;
}

// Trim leading and trailing whitespace
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

void execute_command(char *line) {
    line = trim_space(line);
    if (strlen(line) == 0) return;

    // Dynamically allocate argument array to verify leak-free execution
    char **args = calloc(MAX_ARGS, sizeof(char *));
    if (!args) {
        perror("calloc failed");
        return;
    }

    int arg_count = 0;
    char *token = strtok(line, " \t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t");
    }
}
```

^G Get Help

^X Exit

^O WriteOut

^J Justify

^R Read File

^W Where is

^Y Prev Pg

^V Next Pg

^K Cut Text

^U UnCut Tex

~ -- zsh

...

~ -- nano memory

UW PICO 5.09File: memory_and_test_shell.c

```
args[arg_count] = NULL;

if (arg_count > 0) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
    } else if (pid == 0) {
        execvp(args[0], args);
        perror("Command execution failed");
        free(args); // Clean release in child
        exit(EXIT_FAILURE);
    } else {
        int status;
        waitpid(pid, &status, 0);
    }
}

// Cleanly release allocated memory resources in parent
free(args);
}

int main() {
    char input_buf[BUF_SIZE];
    char *prompt = create_dynamic_prompt("ValgrindShell");

    // Enable automated non-interactive pipeline testing
    int is_interactive = isatty(STDIN_FILENO);

    while (1) {
        if (is_interactive) {
            printf("%s", prompt);
            fflush(stdout);
        }

        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_command(input_buf);
    }

    // Explicitly free dynamic resources before exit to pass Valgrind checks
    free(prompt);
    return 0;
}
```

^G Get Help

^X Exit

^O WriteOut

^J Justify

^R Read File

^W Where is

^Y Prev Pg

^V Next Pg

^K Cut Text

^U UnCut Tex

```

medikondarakesh@Medikondas-MacBook-Air ~ % nano memory_and_test_shell.c
medikondarakesh@Medikondas-MacBook-Air ~ % #!/bin/bash
EXECUTABLE="./memory_and_test_shell"
REPORT_FILE="test_report.txt"

echo "===== " > $REPORT_FILE
echo " AUTOMATED TEST EXECUTION & VERIFICATION " >> $REPORT_FILE
echo "===== " >> $REPORT_FILE
echo "Date: $(date)" >> $REPORT_FILE
echo "" >> $REPORT_FILE

# Test Case 1: Simple Command Execution
echo "Running Test Case 1: Simple Command..."
echo "echo 'Test Case 1 Passed' | $EXECUTABLE > test1_out.tmp 2>&1"
if grep -q "Test Case 1 Passed" test1_out.tmp; then
    echo "[PASS] Test 1: Simple Echo Execution" >> $REPORT_FILE
else
    echo "[FAIL] Test 1: Simple Echo Execution" >> $REPORT_FILE
fi

# Test Case 2: Multi-Argument Command
echo "Running Test Case 2: Multi-Argument Output..."
echo "uname -s" | $EXECUTABLE > test2_out.tmp 2>&1
if [ -s test2_out.tmp ]; then
    echo "[PASS] Test 2: System Command Output Verified" >> $REPORT_FILE
else
    echo "[FAIL] Test 2: System Command Output Empty" >> $REPORT_FILE
fi

# Test Case 3: Exit Cleanly
echo "Running Test Case 3: Exit Processing..."
echo "exit" | $EXECUTABLE > test3_out.tmp 2>&1
if [ $? -eq 0 ]; then
    echo "[PASS] Test 3: Shell Exit Behavior Clean" >> $REPORT_FILE
else
    echo "[FAIL] Test 3: Shell Exit Non-Zero" >> $REPORT_FILE
fi

rm -f test1_out.tmp test2_out.tmp test3_out.tmp
echo "===== " >> $REPORT_FILE
echo "Automated test suite completed. Results appended to $REPORT_FILE"
cat $REPORT_FILE
#!/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
EXECUTABLE="./memory_and_test_shell"
REPORT_FILE="test_report.txt"

echo "===== " > $REPORT_FILE
echo " AUTOMATED TEST EXECUTION & VERIFICATION " >> $REPORT_FILE
echo "===== " >> $REPORT_FILE
echo "Date: $(date)" >> $REPORT_FILE
echo "" >> $REPORT_FILE

# Test Case 1: Simple Command Execution

# Test Case 3: Exit Cleanly
echo "Running Test Case 3: Exit Processing..."
echo "exit" | $EXECUTABLE > test3_out.tmp 2>&1
if [ $? -eq 0 ]; then
    echo "[PASS] Test 3: Shell Exit Behavior Clean" >> $REPORT_FILE
else
    echo "[FAIL] Test 3: Shell Exit Non-Zero" >> $REPORT_FILE
fi

rm -f test1_out.tmp test2_out.tmp test3_out.tmp
echo "===== " >> $REPORT_FILE
echo "Automated test suite completed. Results appended to $REPORT_FILE"
cat $REPORT_FILE
zsh: no such file or directory: #/bin/bash
zsh: command not found: #
Running Test Case 1: Simple Command...
zsh: command not found: #
Running Test Case 2: Multi-Argument Output...
zsh: command not found: #
Running Test Case 3: Exit Processing...
Automated test suite completed. Results appended to test_report.txt
=====
AUTOMATED TEST EXECUTION & VERIFICATION
=====
Date: Fri Oct 2 13:08:06 IST 2026

[FAIL] Test 1: Simple Echo Execution
[PASS] Test 2: System Command Output Verified
[FAIL] Test 3: Shell Exit Non-Zero
=====
medikondarakesh@Medikondas-MacBook-Air ~ % nano memory_and_test_shell.c
gcc -g -Wall memory_and_test_shell.c -o memory_and_test_shell
nano run_tests.sh
chmod +x run_tests.sh
./run_tests.sh

```